

UNIT-III – (2ND HALF)

Clustering

- 1.Partitional Clustering
- 2.Hierarchical Clustering
- 3.Density based Clustering
- 4.The BIRCH Algorithm
- 5.The CURE Algorithm

What is Clustering?

- ❖ Clustering, also known as cluster analysis, groups data points based on their similarity with each other. Unlike supervised learning, where we have a target variable, clustering works with unlabelled data points. Its goal is to form groups of homogeneous data points from a heterogeneous dataset.
- ❖ Similarity is evaluated using metrics like Euclidean distance, Cosine similarity, or Manhattan distance. The resulting clusters can have arbitrary shapes, not necessarily circular.
- ❖ Clustering or cluster analysis is a machine learning technique, which groups the unlabelled dataset. It can be defined as ***"A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group."***
- ❖ It does it by finding some similar patterns in the unlabelled dataset such as shape, size, color, behavior, etc., and divides them as per the presence and absence of those similar patterns.
- ❖ It is an unsupervised learning method, hence no supervision is provided to the algorithm, and it deals with the unlabeled dataset.
- ❖ After applying this clustering technique, each cluster or group is provided with a cluster-ID. ML system can use this id to simplify the processing of large and complex datasets.

- **Types of Clustering:**

- 1. **Hard Clustering:**

- Each data point belongs to a cluster completely or not. For instance, if we have four data points and want to cluster them into two clusters, each point will belong to either cluster 1 or cluster 2.
 - Example:
 - Data Points: A, B, C, D
 - Clusters: A (C1), B (C2), C (C2), D (C1)

- 2. **Soft Clustering:**

- Instead of assigning data points to separate clusters, we evaluate the probability of a point belonging to each cluster.
 - Example:
 - Data Points: A, B, C, D
 - Probabilities:
 - A: C1 (0.91), C2 (0.09)
 - B: C1 (0.3), C2 (0.7)
 - C: C1 (0.17), C2 (0.83)
 - D: C1 (1), C2 (0)

Example: Let's understand the clustering technique with the real-world example of Mall: When we visit any shopping mall, we can observe that the things with similar usage are grouped together. Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things. The clustering technique also works in the same way. Other examples of clustering are grouping documents according to the topic.

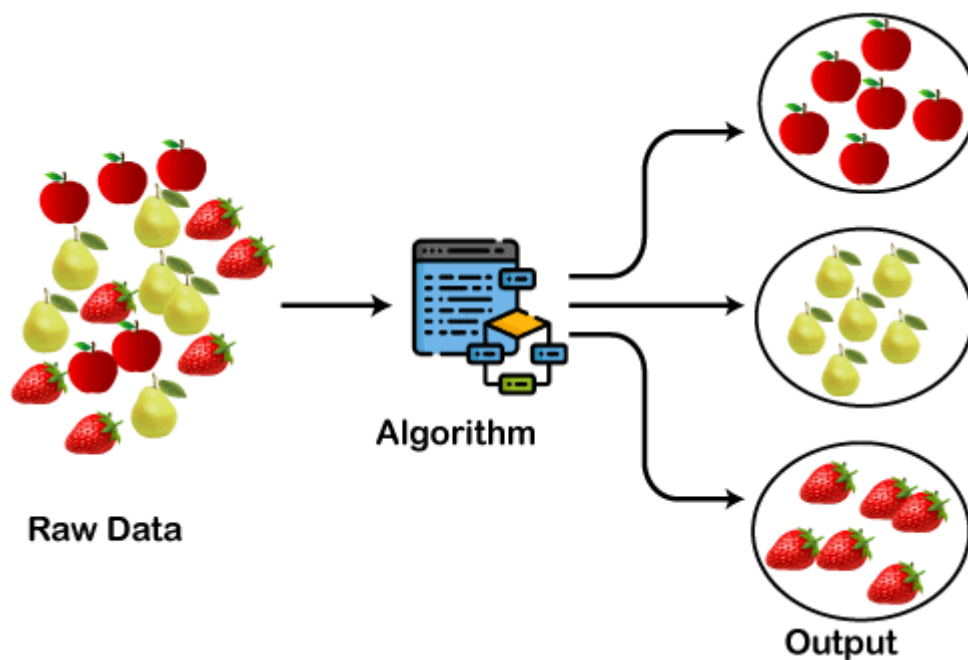
The clustering technique can be widely used in various tasks. Some most common uses of this technique are:

- Market Segmentation
 - Statistical data analysis
 - Social network analysis

- Image segmentation
- Anomaly detection, etc.

Apart from these general usages, it is used by the **Amazon** in its recommendation system to provide the recommendations as per the past search of products. **Netflix** also uses this technique to recommend the movies and web-series to its users as per the watch history.

The below diagram explains the working of the clustering algorithm. We can see the different fruits are divided into several groups with similar properties.



Clustering Algorithms

The Clustering algorithms can be divided based on their models that are explained above. There are different types of clustering algorithms published, but only a few are commonly used. The clustering algorithm is based on the kind of data that we are using. Such as, some algorithms need to guess the number of clusters in the given dataset, whereas some are required to find the minimum distance between the observation of the dataset.

1. **K-Means algorithm:** The k-means algorithm is one of the most popular clustering algorithms. It classifies the dataset by dividing the samples into different clusters of equal variances. The number of clusters must be specified in this algorithm. It is fast with fewer computations required, with the linear complexity of **O(n)**.

2. **Mean-shift algorithm:** Mean-shift algorithm tries to find the dense areas in the smooth density of data points. It is an example of a centroid-based model, that works on updating the candidates for centroid to be the center of the points within a given region.
3. **DBSCAN Algorithm:** It stands for **Density-Based Spatial Clustering of Applications with Noise**. It is an example of a density-based model similar to the mean-shift, but with some remarkable advantages. In this algorithm, the areas of high density are separated by the areas of low density. Because of this, the clusters can be found in any arbitrary shape.
4. **Expectation-Maximization Clustering using GMM:** This algorithm can be used as an alternative for the k-means algorithm or for those cases where K-means can be failed. In GMM, it is assumed that the data points are Gaussian distributed.
5. **Agglomerative Hierarchical algorithm:** The Agglomerative hierarchical algorithm performs the bottom-up hierarchical clustering. In this, each data point is treated as a single cluster at the outset and then successively merged. The cluster hierarchy can be represented as a tree-structure.
6. **Affinity Propagation:** It is different from other clustering algorithms as it does not require to specify the number of clusters. In this, each data point sends a message between the pair of data points until convergence. It has $O(N^2T)$ time complexity, which is the main drawback of this algorithm.

Applications of Clustering

Below are some commonly known applications of clustering technique in Machine Learning:

- **In Identification of Cancer Cells:** The clustering algorithms are widely used for the identification of cancerous cells. It divides the cancerous and non-cancerous data sets into different groups.
- **In Search Engines:** Search engines also work on the clustering technique. The search result appears based on the closest object to the search query. It does it by grouping similar data objects in one group that is far from the other dissimilar objects. The accurate result of a query depends on the quality of the clustering algorithm used.
- **Customer Segmentation:** It is used in market research to segment the customers based on their choice and preferences.

- **In Biology:** It is used in the biology stream to classify different species of plants and animals using the image recognition technique.
- **In Land Use:** The clustering technique is used in identifying the area of similar lands use in the GIS database. This can be very useful to find that for what purpose the particular land should be used, that means for which purpose it is more suitable.

1.Partitional Clustering:

Partitional clustering is a type of clustering technique that divides a dataset into non-overlapping subsets or clusters, such that each data point belongs to exactly one cluster. The main idea is to partition the data into a predefined number of clusters, which can be done using various algorithms. Here, we will discuss some key partitional clustering algorithms and their characteristics:

1. K-means Clustering

Algorithm

1. **Initialization:** Randomly select KKK initial centroids from the dataset.
2. **Assignment:** Assign each data point to the nearest centroid based on a distance metric (usually Euclidean distance).
3. **Update:** Calculate the new centroids by taking the mean of all the data points assigned to each centroid.
4. **Repeat:** Repeat the assignment and update steps until the centroids no longer change significantly or a maximum number of iterations is reached.

Characteristics

- **Pros:**
 - Simple and easy to implement.
 - Efficient for large datasets.
 - Works well when clusters are globular and well-separated.
- **Cons:**
 - Requires the number of clusters KKK to be specified in advance.
 - Sensitive to the initial placement of centroids.
 - Can converge to a local minimum.
 - Not suitable for non-spherical clusters or clusters of varying sizes and densities.

2. K-medoids Clustering

Algorithm

1. **Initialization:** Randomly select KKK initial medoids (representative points from the dataset).
2. **Assignment:** Assign each data point to the nearest medoid.
3. **Update:** For each cluster, select a new medoid such that the sum of distances from the medoid to all points in the cluster is minimized.
4. **Repeat:** Repeat the assignment and update steps until the medoids no longer change significantly or a maximum number of iterations is reached.

Characteristics

- **Pros:**
 - More robust to noise and outliers compared to K-means.
 - Suitable for clusters with different shapes and sizes.
- **Cons:**
 - More computationally intensive than K-means, especially for large datasets.
 - Requires the number of clusters KKK to be specified in advance.

3. CLARA (Clustering Large Applications)

CLARA is an extension of K-medoids designed to handle larger datasets by sampling.

Algorithm

1. **Sampling:** Randomly select multiple subsets of the dataset.
2. **K-medoids on Sample:** Apply K-medoids clustering on each subset.
3. **Best Clustering:** Choose the best clustering result based on a quality metric (e.g., average dissimilarity).
4. **Assignment:** Assign all data points to the nearest medoid of the best clustering.

Characteristics

- **Pros:**
 - Handles larger datasets better than K-medoids by working on samples.
- **Cons:**
 - Quality of clustering depends on the quality of the samples.

4. Fuzzy C-means Clustering

Algorithm

1. **Initialization:** Initialize the membership matrix randomly.
2. **Membership Assignment:** Assign membership values to each data point for each cluster based on the distance to the centroids and a fuzziness parameter.
3. **Update Centroids:** Calculate new centroids based on the membership values.
4. **Repeat:** Repeat the membership assignment and update steps until convergence.

Characteristics

- **Pros:**
 - Allows data points to belong to multiple clusters with varying degrees of membership.
 - Suitable for overlapping clusters.
- **Cons:**
 - Requires the number of clusters and the fuzziness parameter to be specified in advance.
 - More computationally intensive than K-means.

Comparison of Partitional Clustering Algorithms

- **K-means:** Simple and efficient, but sensitive to initial conditions and outliers.
- **K-medoids:** More robust to outliers but computationally intensive.
- **CLARA:** Scales better for large datasets but depends on sample quality.
- **Fuzzy C-means:** Handles overlapping clusters but requires additional parameters.

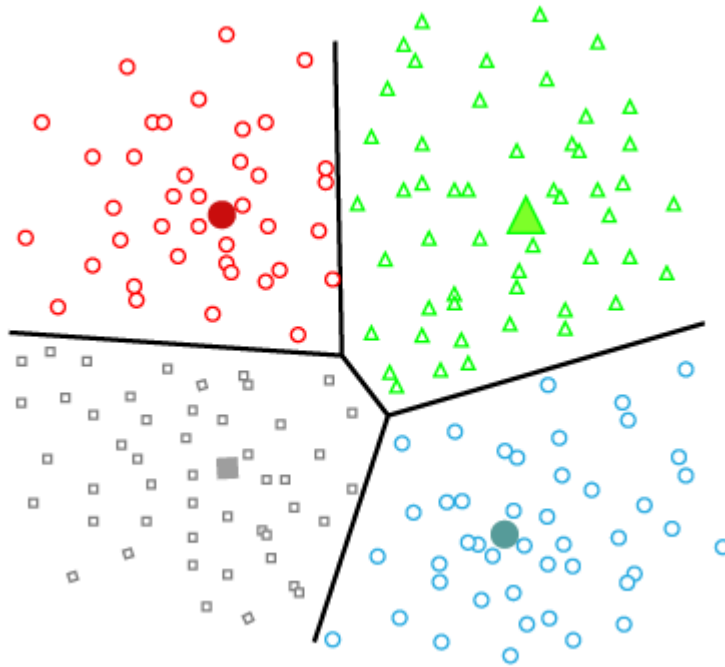
Use Cases

- **K-means:** Customer segmentation, document clustering, image compression.
- **K-medoids:** Gene expression data analysis, clustering of geographic data.
- **CLARA:** Large-scale data mining applications.
- **Fuzzy C-means:** Image segmentation, pattern recognition, medical diagnosis.

❖ It is a type of clustering that divides the data into non-hierarchical groups. It is also known as the centroid-based method. The most

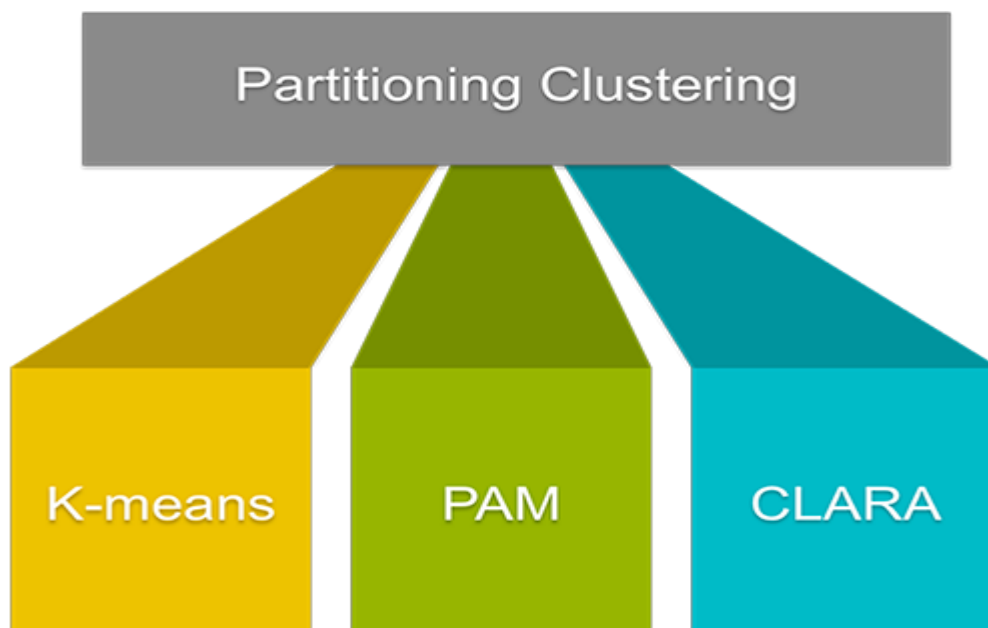
common example of partitioning clustering is the K-Means Clustering algorithm.

- ❖ In this type, the dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups. The cluster center is created in such a way that the distance between the data points of one cluster is minimum as compared to another cluster centroid.



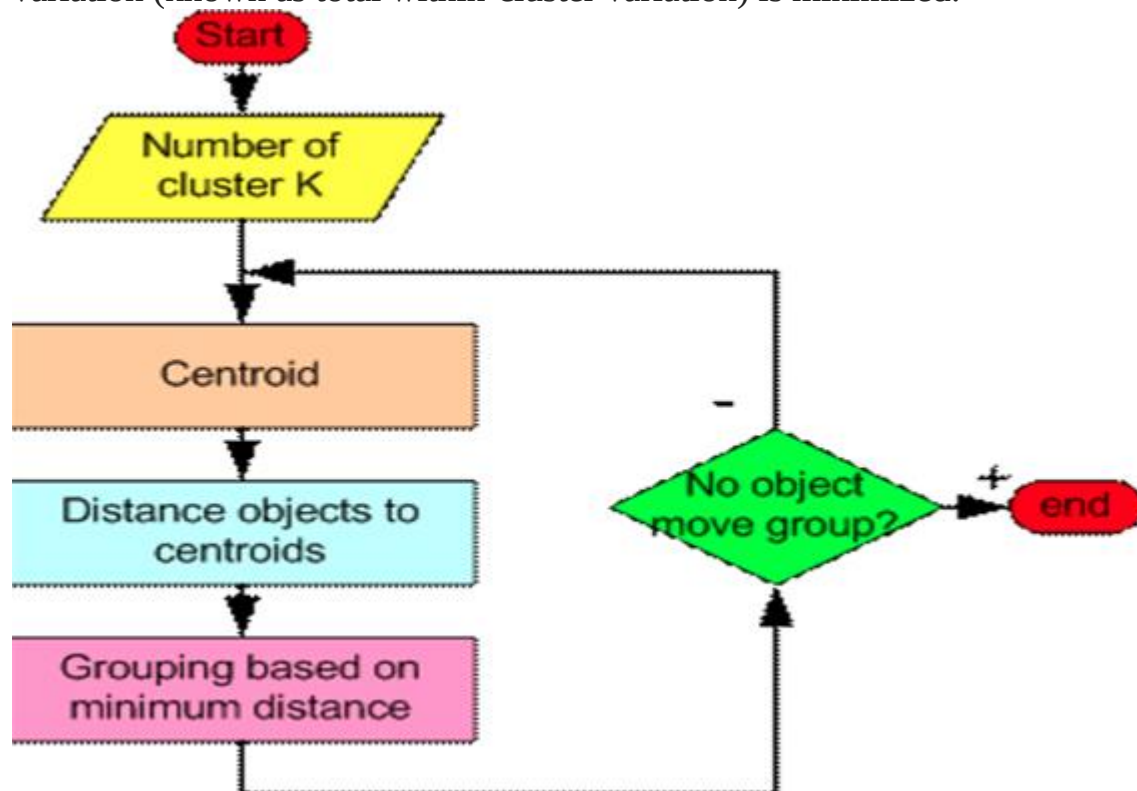
Clustering in Machine Learning

Partitioning Algorithms used in Clustering -



Types of Partitional Clustering

K-Means Algorithm (A centroid based Technique): It is one of the most commonly used algorithm for partitioning a given data set into a set of k groups (i.e. k clusters), where k represents the number of groups. It classifies objects in multiple groups (i.e., clusters), such that objects within the same cluster are as similar as possible (i.e., high *intra-class similarity*), whereas objects from different clusters are as dissimilar as possible (i.e., low *inter-class similarity*). In k-means clustering, each cluster is represented by its center (i.e, *centroid*) which corresponds to the mean of points assigned to the cluster. The basic idea behind k-means clustering consists of defining clusters so that the total intra-cluster variation (known as total within-cluster variation) is minimized.



Process for K-means Algorithm

Steps involved in K-Means Clustering :

1. The first step when using k-means clustering is to indicate the number of clusters (k) that will be generated in the final solution.
2. The algorithm starts by randomly selecting k objects from the data set to serve as the initial centers for the clusters. The selected objects are also known as cluster means or centroids.
3. Next, each of the remaining objects is assigned to its closest centroid, where closest is defined using the [Euclidean distance](#) between the object and the cluster mean. This step is called “cluster assignment step”.
4. After the assignment step, the algorithm computes the new mean value of each cluster. The term cluster “centroid update” is used to design this step. Now that the centers have been recalculated, every observation is checked again to see if it might be closer to a different cluster. All the objects are reassigned again using the updated cluster means.
5. The cluster assignment and centroid update steps are iteratively repeated until the cluster assignments stop changing (i.e until *convergence* is achieved). That is, the clusters formed in the current iteration are the same as those obtained in the previous iteration.

Step 1 ->

Subject	A	B
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Step 2 ->

	Individual	Mean Vector (centroid)
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Step 3 ->

	Cluster 1		Cluster 2	
Step	Individual	Mean Vector (centroid)	Individual	Mean Vector (centroid)
1	1	(1.0, 1.0)	4	(5.0, 7.0)
2	1, 2	(1.2, 1.5)	4	(5.0, 7.0)
3	1, 2, 3	(1.8, 2.3)	4	(5.0, 7.0)
4	1, 2, 3	(1.8, 2.3)	4, 5	(4.2, 6.0)
5	1, 2, 3	(1.8, 2.3)	4, 5, 6	(4.3, 5.7)
6	1, 2, 3	(1.8, 2.3)	4, 5, 6, 7	(4.1, 5.4)

Step 4 ->

	Individual	Mean Vector (centroid)
Cluster 1	1, 2, 3	(1.8, 2.3)
Cluster 2	4, 5, 6, 7	(4.1, 5.4)

Step 5 ->

Individual	Distance to mean (centroid) of Cluster 1	Distance to mean (centroid) of Cluster 2
1	1.5	5.4
2	0.4	4.3
3	2.1	1.8
4	5.7	1.8
5	3.2	0.7
6	3.8	0.6
7	2.8	1.1

Step 6 ->

	Individual	Mean Vector (centroid)
Cluster 1	1, 2	(1.3, 1.5)
Cluster 2	3, 4, 5, 6, 7	(3.9, 5.1)

Example for K-Means Clustering

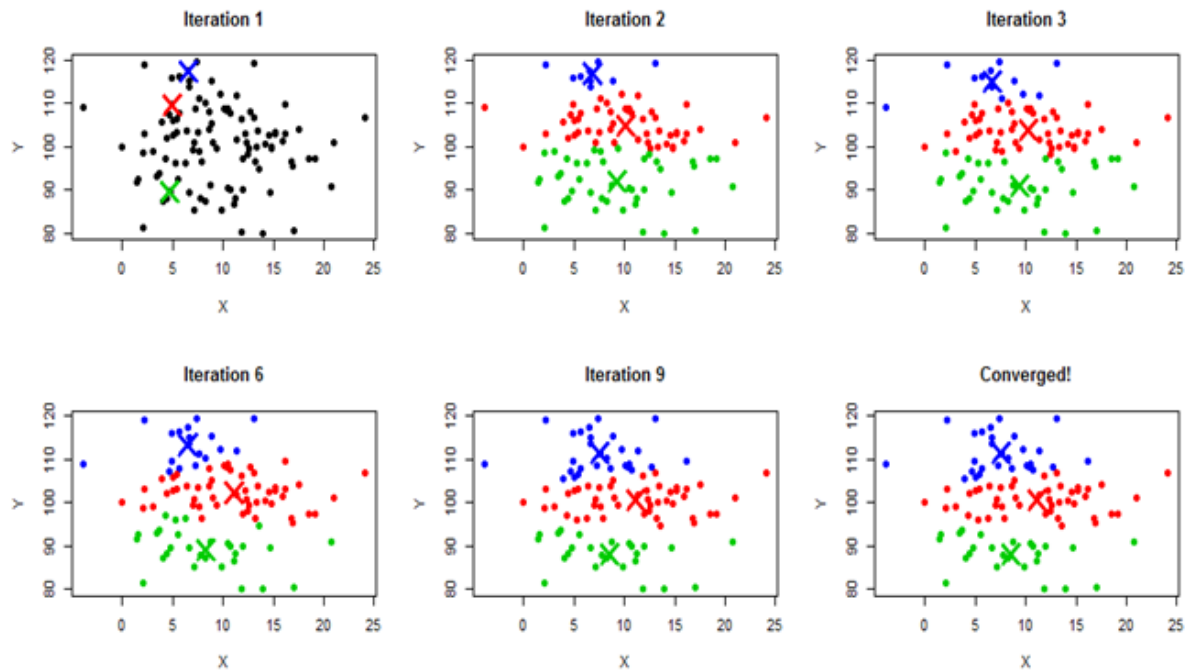
Step 5 ->

Individual	Distance to mean (centroid) of Cluster 1	Distance to mean (centroid) of Cluster 2
1	1.5	5.4
2	0.4	4.3
3	2.1	1.8
4	5.7	1.8
5	3.2	0.7
6	3.8	0.6
7	2.8	1.1

Step 6 ->

	Individual	Mean Vector (centroid)
Cluster 1	1, 2	(1.3, 1.5)
Cluster 2	3, 4, 5, 6, 7	(3.9, 5.1)

Example for K-Means Clustering



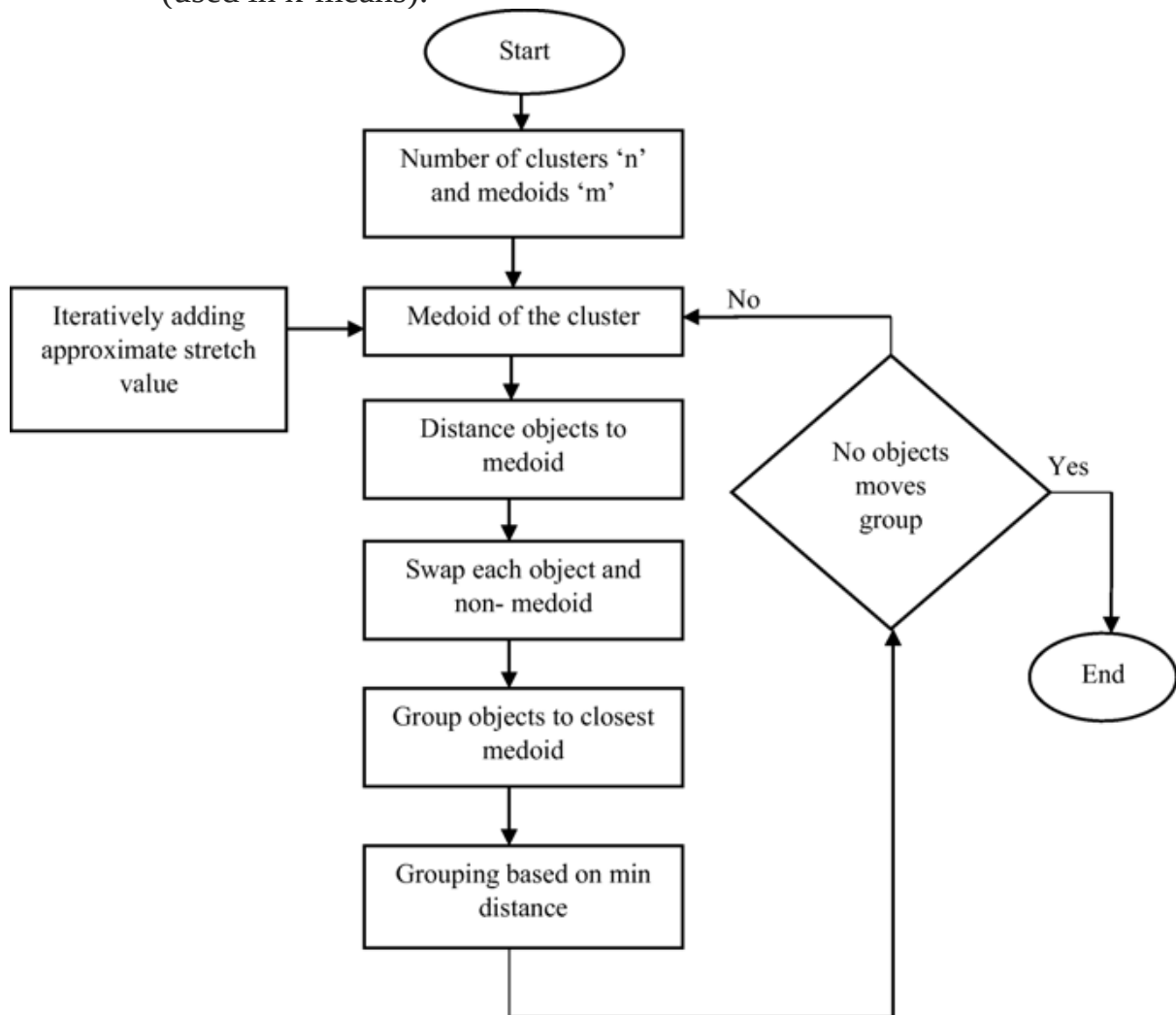
Plotting of K-means Clustering

K-Medoids Algorithm (Partitioning Around Medoid) :

1. A medoid can be defined as the point in the cluster, whose similarities with all the other points in the cluster is maximum.
2. In k-medoids clustering, each cluster is represented by one of the data point in the cluster. These points are named cluster medoids. The term medoid refers to an object within a cluster for which average dissimilarity between it and all the other the members of the cluster is minimal. It corresponds to the most centrally located point in the cluster.
3. These objects (one per cluster) can be considered as a representative example of the members of that cluster which may be useful in some situations. Recall that, in k-means clustering, the center of a given

cluster is calculated as the mean value of all the data points in the cluster.

4. K-medoid is a robust alternative to k-means clustering. This means that, the algorithm is less sensitive to noise and outliers, compared to k-means, because it uses medoids as cluster centers instead of means (used in k-means).



Steps involved in K-Medoid Clustering

Steps involved in K-Medoids Clustering :

1. The PAM algorithm is based on the search for k representative objects or medoids among the observations of the data set.

2. After finding a set of k medoids, clusters are constructed by assigning each observation to the nearest medoid.
3. Next, each selected medoid m and each non-medoid data point are swapped and the objective function is computed. The objective function corresponds to the sum of the dissimilarities of all objects to their nearest medoid.
4. The SWAP step attempts to improve the quality of the clustering by exchanging selected objects (medoids) and non-selected objects. If the objective function can be reduced by interchanging a selected object with an unselected object, then the swap is carried out. This is continued until the objective function can no longer be decreased. The goal is to find k representative objects which minimize the sum of the dissimilarities of the observations to their closest representative object.

2. Hierarchical Clustering

Hierarchical clustering is another unsupervised machine learning algorithm, which is used to group the unlabeled datasets into a cluster and also known as **hierarchical cluster analysis** or HCA.

In this algorithm, we develop the hierarchy of clusters in the form of a tree, and this tree-shaped structure is known as the **dendrogram**.

Sometimes the results of K-means clustering and hierarchical clustering may look similar, but they both differ depending on how they work. As there is no requirement to predetermine the number of clusters as we did in the K-Means algorithm.

The hierarchical clustering technique has two approaches:

1. **Agglomerative:** Agglomerative is a **bottom-up** approach, in which the algorithm starts with taking all data points as single clusters and merging them until one cluster is left.

2. **Divisive:** Divisive algorithm is the reverse of the agglomerative algorithm as it is a **top-down approach**.

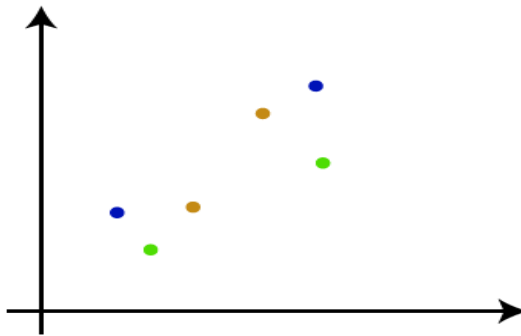
Agglomerative Hierarchical clustering

The agglomerative hierarchical clustering algorithm is a popular example of HCA. To group the datasets into clusters, it follows the **bottom-up approach**. It means, this algorithm considers each dataset as a single cluster at the beginning, and then start combining the closest pair of clusters together. It does this until all the clusters are merged into a single cluster that contains all the datasets.

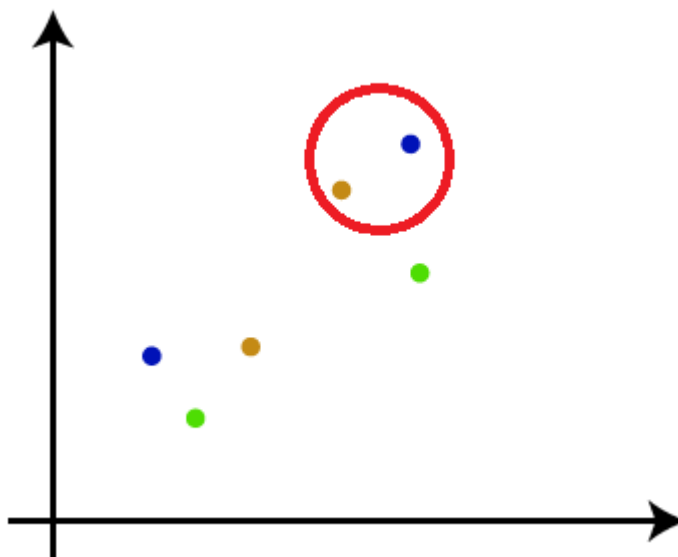
How the Agglomerative Hierarchical clustering Work?

The working of the AHC algorithm can be explained using the below steps:

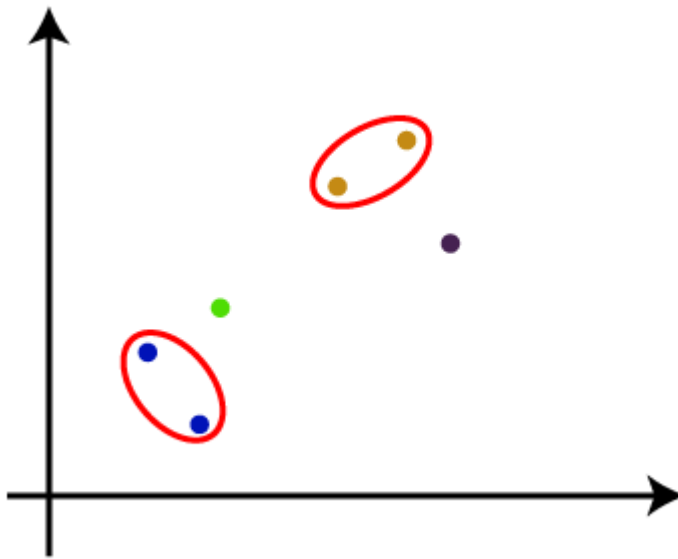
- **Step-1:** Create each data point as a single cluster. Let's say there are N data points, so the number of clusters will also be N .



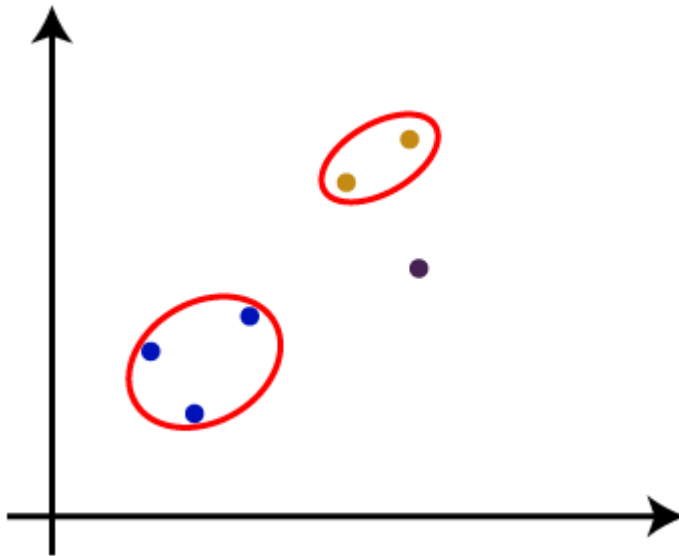
- **Step-2:** Take two closest data points or clusters and merge them to form one cluster. So, there will now be $N-1$ clusters.

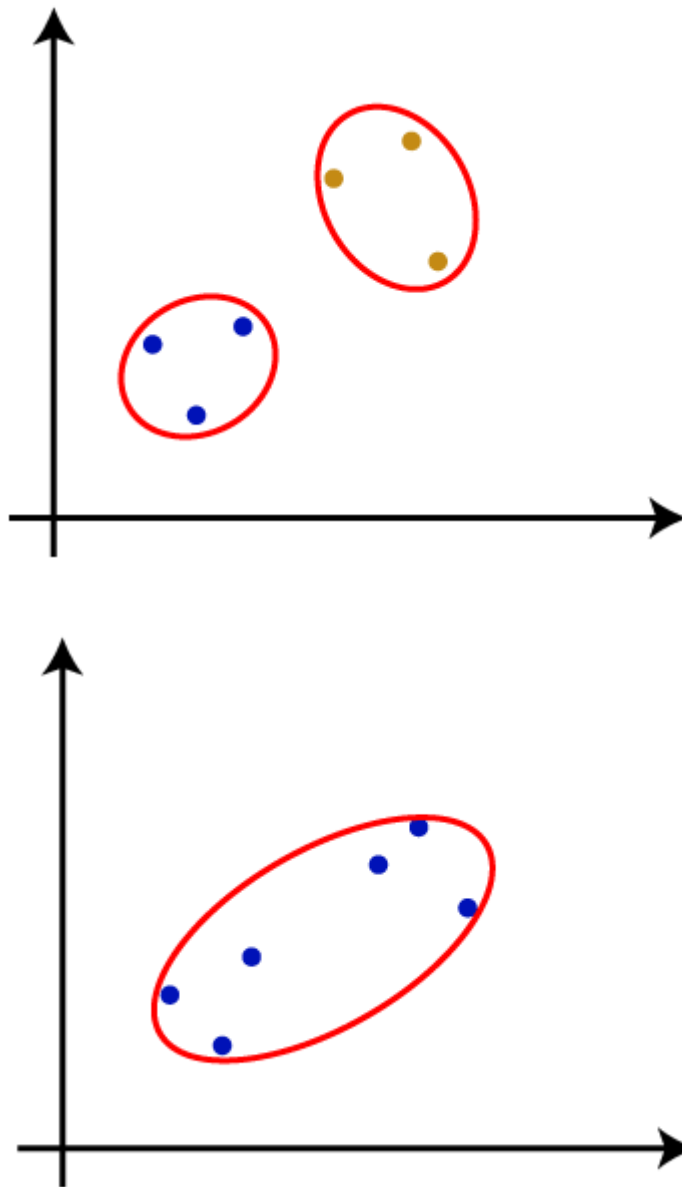


- **Step-3:** Again, take the two closest clusters and merge them together to form one cluster. There will be $N-2$ clusters.



- **Step-4:** Repeat Step 3 until only one cluster left. So, we will get the following clusters. Consider the below images:



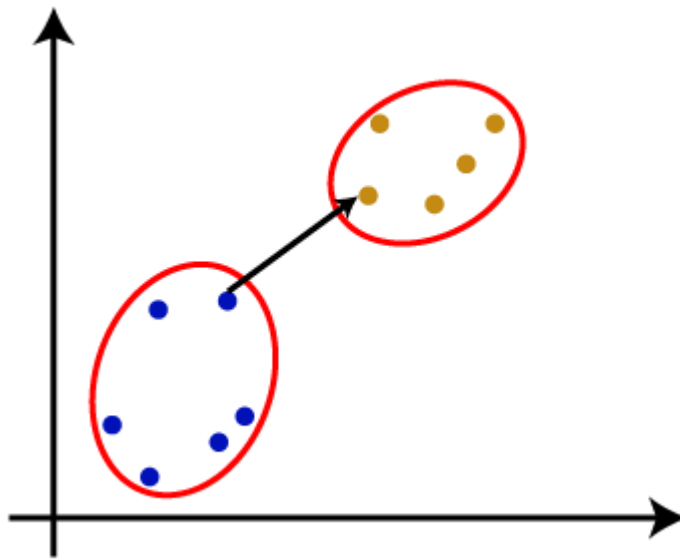


- **Step-5:** Once all the clusters are combined into one big cluster, develop the dendrogram to divide the clusters as per the problem.

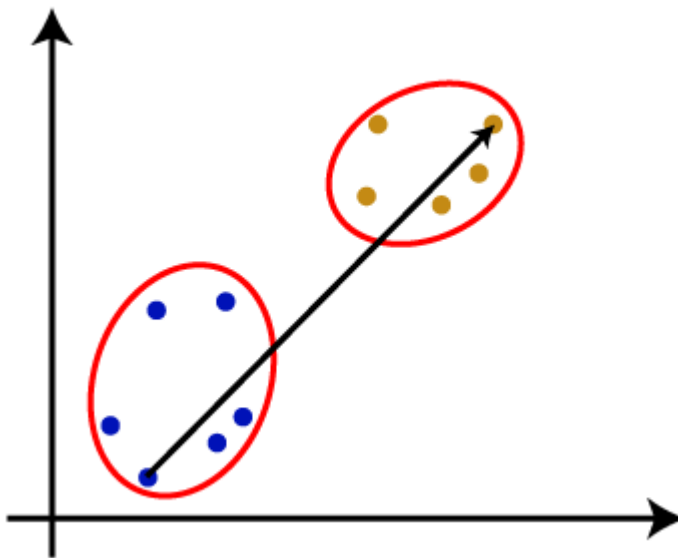
Measure for the distance between two clusters

As we have seen, the **closest distance** between the two clusters is crucial for the hierarchical clustering. There are various ways to calculate the distance between two clusters, and these ways decide the rule for clustering. These measures are called **Linkage methods**. Some of the popular linkage methods are given below:

1. **Single Linkage:** It is the Shortest Distance between the closest points of the clusters. Consider the below image:

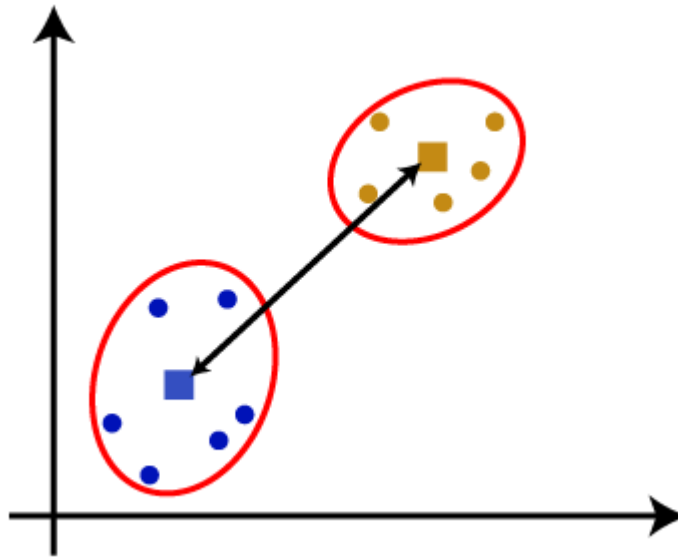


2. **Complete Linkage:** It is the farthest distance between the two points of two different clusters. It is one of the popular linkage methods as it forms tighter clusters than single-linkage.



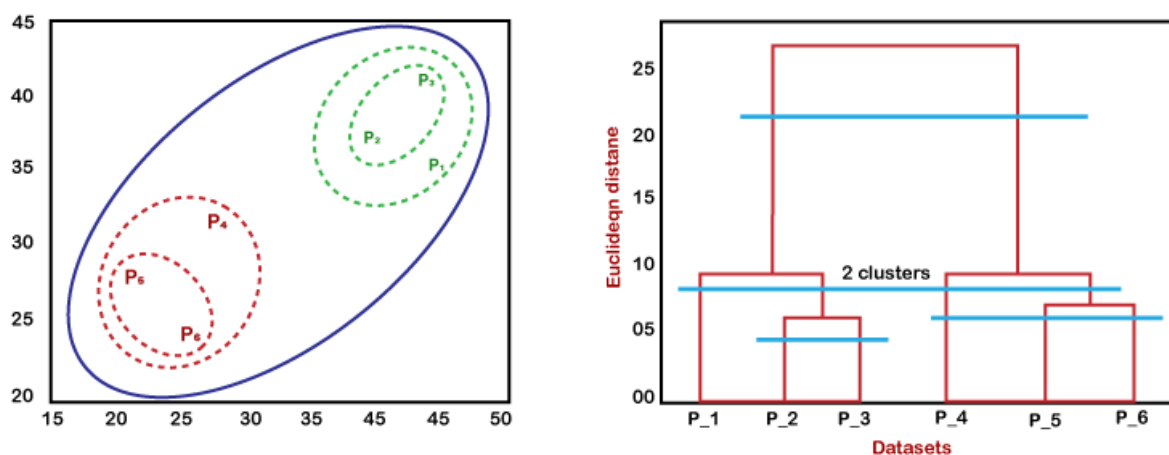
3. **Average Linkage:** It is the linkage method in which the distance between each pair of datasets is added up and then divided by the total number of datasets to calculate the average distance between two clusters. It is also one of the most popular linkage methods.

4. **Centroid Linkage:** It is the linkage method in which the distance between the centroid of the clusters is calculated. Consider the below image:



Working of Dendrogram in Hierarchical clustering

The dendrogram is a tree-like structure that is mainly used to store each step as a memory that the HC algorithm performs. In the dendrogram plot, the Y-axis shows the Euclidean distances between the data points, and the x-axis shows all the data points of the given dataset.



In the above diagram, the left part is showing how clusters are created in agglomerative clustering, and the right part is showing the corresponding dendrogram.

- firstly, the datapoints P₂ and P₃ combine together and form a cluster, correspondingly a dendrogram is created, which connects P₂ and P₃ with

a rectangular shape. The height is decided according to the Euclidean distance between the data points.

- In the next step, P5 and P6 form a cluster, and the corresponding dendrogram is created. It is higher than of previous, as the Euclidean distance between P5 and P6 is a little bit greater than the P2 and P3.
- Again, two new dendrograms are created that combine P1, P2, and P3 in one dendrogram, and P4, P5, and P6, in another dendrogram.
- At last, the final dendrogram is created that combines all the data points together.

Steps for implementation of AHC using Python:

The steps for implementation will be the same as the k-means clustering, except for some changes such as the method to find the number of clusters. Below are the steps:

1. **Data Pre-processing**
2. **Finding the optimal number of clusters using the Dendrogram**
3. **Training the hierarchical clustering model**
4. **Visualizing the clusters**

3.Density based Clustering

Density-Based Clustering refers to one of the most popular unsupervised learning methodologies used in model building and machine learning algorithms. The data points in the region separated by two clusters of low point density are considered as noise. The surroundings with a radius ϵ of a given object are known as the ϵ neighborhood of the object. If the ϵ neighborhood of the object comprises at least a minimum number, MinPts of objects, then it is called a core object.

1. Clusters and Noise:

- DBSCAN identifies clusters as dense regions in the data space, separated by areas of lower point density (noise).
- Unlike partitioning methods (e.g., K-means) that work well for spherical or convex clusters, DBSCAN handles arbitrary-shaped clusters and noise effectively.

2. Parameters:

- **Eps (ϵ):** Defines the neighborhood around a data point. If the distance between two points is less than or equal to ϵ , they are considered neighbors.
- **MinPts:** Minimum number of neighbors within ϵ radius. It's derived from the number of dimensions in the dataset ($\text{MinPts} \geq D+1$, where D is the number of dimensions).

3. Types of Data Points in DBSCAN:

- **Core Point:** Has more than MinPts neighbors within ϵ .
- **Border Point:** Has fewer neighbors than MinPts but is reachable from a core point.
- **Noise Point:** Neither core nor border point.

Density-Based Clustering - Background

There are two different parameters to calculate the density-based clustering

E_{ps} : It is considered as the maximum radius of the neighborhood.

MinPts: MinPts refers to the minimum number of points in an E_{ps} neighborhood of that point.

$NE_{ps}(i) : \{ k \text{ belongs to } D \text{ and } \text{dist}(i,k) \leq E_{ps} \}$

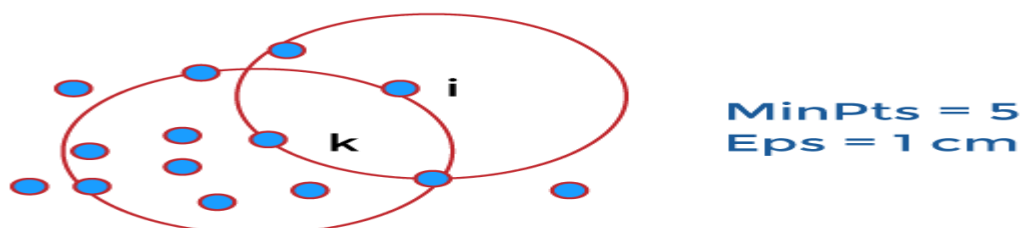
Directly density reachable:

A point i is considered as the directly density reachable from a point k with respect to E_{ps} , MinPts if

i belongs to $NE_{ps}(k)$

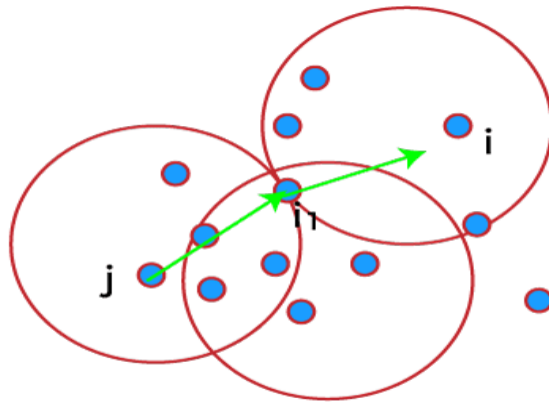
Core point condition:

$NE_{ps}(k) \geq \text{MinPts}$



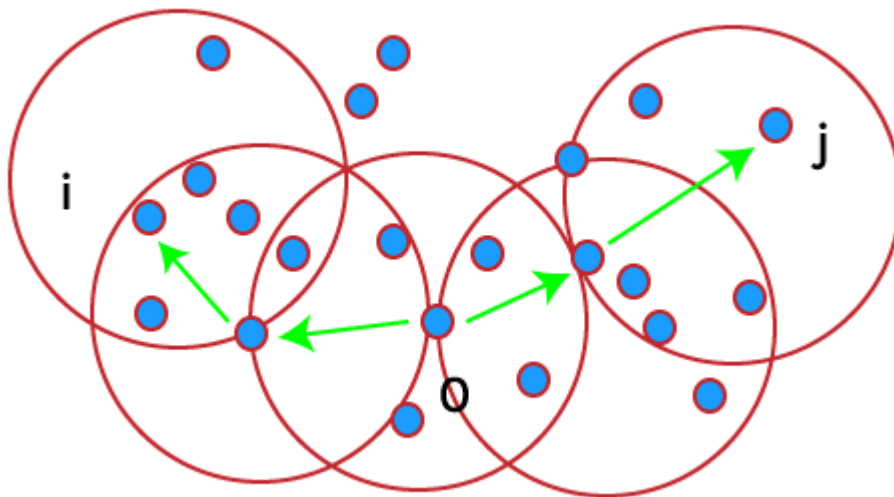
Density reachable:

A point denoted by i is a density reachable from a point j with respect to Eps , $MinPts$ if there is a sequence chain of a point $i_1, \dots, i_n$, $i_1 = j$, $i_n = i$ such that i_{i+1} is directly density reachable from i_i .



Density connected:

A point i refers to density connected to a point j with respect to Eps , $MinPts$ if there is a point o such that both i and j are considered as density reachable from o with respect to Eps and $MinPts$.



Working of Density-Based Clustering

Suppose a set of objects is denoted by D' , we can say that an object I is directly density reachable from the object j only if it is located within the ϵ neighborhood of j , and j is a core object.

An object i is density reachable from the object j with respect to ϵ and MinPts in a given set of objects, D' only if there is a sequence of object chains point $i_1, \dots, i_n$, $i_1 = j$, $i_n = i$ such that i_{i+1} is directly density reachable from i_i with respect to ϵ and MinPts.

An object i is density connected object j with respect to ϵ and MinPts in a given set of objects, D' only if there is an object o belongs to D such that both point i and j are density reachable from o with respect to ϵ and MinPts.

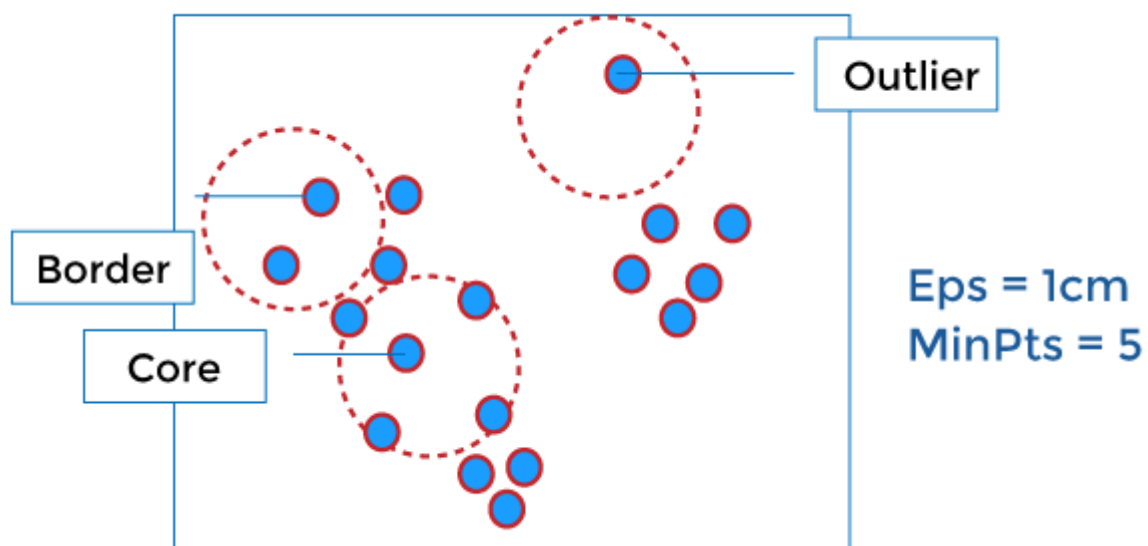
Major Features of Density-Based Clustering

The primary features of Density-based clustering are given below.

- It is a scan method.
- It requires density parameters as a termination condition.
- It is used to manage noise in data clusters.
- Density-based clustering is used to identify clusters of arbitrary size.

Density-Based Clustering Methods

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. It depends on a density-based notion of cluster. It also identifies clusters of arbitrary size in the spatial database with outliers.



OPTICS

OPTICS stands for Ordering Points To Identify the Clustering Structure. It gives a significant order of database with respect to its density-based clustering

structure. The order of the cluster comprises information equivalent to the density-based clustering related to a long range of parameter settings. OPTICS methods are beneficial for both automatic and interactive cluster analysis, including determining an intrinsic clustering structure.

4. BIRCH algorithm

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) is an efficient data clustering algorithm designed to handle large datasets. It incrementally and dynamically clusters incoming multi-dimensional metric data points to produce a high-quality clustering result with a single scan of the dataset.

Its inventors claim BIRCH to be the "first clustering algorithm proposed in the database area to handle 'noise' (data points that are not part of the underlying pattern) effectively", beating DBSCAN by two months. The BIRCH algorithm received the SIGMOD 10 year test of time award in 2006.

Basic clustering algorithms like K means and agglomerative clustering are the most commonly used clustering algorithms. But when performing clustering on very large datasets, BIRCH and DBSCAN are the advanced clustering algorithms useful for performing precise clustering on large datasets. Moreover, BIRCH is very useful because of its easy implementation. BIRCH is a clustering algorithm that clusters the dataset first in small summaries, then after small summaries get clustered. It does not directly cluster the dataset.

Stages of BIRCH

BIRCH is often used to complement other clustering algorithms by creating a summary of the dataset that the other clustering algorithm can now use. However, BIRCH has one major drawback it can only process metric attributes. A metric attribute is an attribute whose values can be represented in Euclidean space, i.e., no categorical attributes should be present. The BIRCH clustering algorithm consists of two stages:

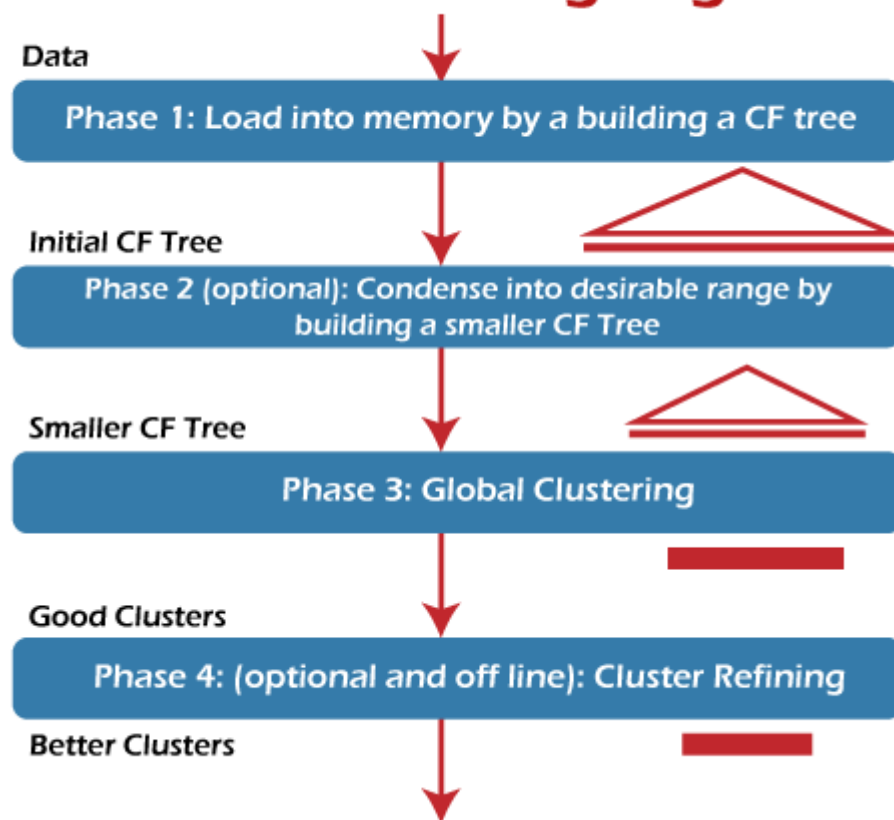
1. **Building the CF Tree:** BIRCH summarizes large datasets into smaller, dense regions called Clustering Feature (CF) entries. Formally, a Clustering Feature entry is defined as an ordered triple (N, LS, SS) where 'N' is the number of data points in the cluster, 'LS' is the linear sum of the data points, and 'SS' is the squared sum of the data points in the cluster. A CF entry can be composed of other CF entries. Optionally, we can condense this initial CF tree into a smaller CF.
2. **Global Clustering:** Applies an existing clustering algorithm on the leaves of the CF tree. A CF tree is a tree where each leaf node contains a sub-

cluster. Every entry in a CF tree contains a pointer to a child node, and a CF entry made up of the sum of CF entries in the child nodes. Optionally, we can refine these clusters.

Algorithm

The tree structure of the given data is built by the BIRCH algorithm called the Clustering feature tree (CF tree). This algorithm is based on the CF (clustering features) tree. In addition, this algorithm uses a tree-structured summary to create clusters.

The BIRCH Clustering Algorithm



In context to the CF tree, the algorithm compresses the data into the sets of CF nodes. Those nodes that have several sub-clusters can be called CF subclusters. These CF subclusters are situated in non-terminal CF nodes.

The CF tree is a height-balanced tree that gathers and manages clustering features and holds necessary information of given data for further hierarchical clustering. This prevents the need to work with whole data given as input. The tree cluster of data points as CF is represented by three numbers (N, LS, SS).

- N = number of items in subclusters
- LS = vector sum of the data points
- SS = sum of the squared data points

There are mainly four phases which are followed by the algorithm of BIRCH.

- Scanning data into memory.
- Condense data (resize data).
- Global clustering.
- Refining clusters.

Cluster Features

BIRCH clustering achieves its high efficiency by clever use of a small set of summary statistics to represent a larger set of data points. These summary statistics constitute a CF and represent a sufficient substitute for the actual data for clustering purposes.

A CF is a set of three summary statistics representing a set of data points in a single cluster. These statistics are as follows:

- Count [The number of data values in the cluster]
- Linear Sum [The sum of the individual coordinates. This is a measure of the location of the cluster]
- Squared Sum [The sum of the squared coordinates. This is a measure of the spread of the cluster]

CF Tree

The building process of the CF Tree can be summarized in the following steps, such as:

Step 1: For each given record, BIRCH compares the location of that record with the location of each CF in the root node, using either the linear sum or the mean of the CF. BIRCH passes the incoming record to the root node CF closest to the incoming record.

Step 2: The record then descends down to the non-leaf child nodes of the root node CF selected in step 1. BIRCH compares the location of the record with the location of each non-leaf CF. BIRCH passes the incoming record to the non-leaf node CF closest to the incoming record.

Step 3: The record then descends down to the leaf child nodes of the non-leaf node CF selected in step 2. BIRCH compares the location of the record with the location of each leaf. BIRCH tentatively passes the incoming record to the leaf closest to the incoming record.

Step 4: Perform one of the below points (i) or (ii):

1. If the radius of the chosen leaf, including the new record, does not exceed the threshold T , then the incoming record is assigned to that leaf. The leaf and its parent CF's are updated to account for the new data point.
2. If the radius of the chosen leaf, including the new record, exceeds the Threshold T , then a new leaf is formed, consisting of the incoming record only. The parent CFs is updated to account for the new data point.

If step 4(ii) is executed, and the maximum L leaves are already in the leaf node, the leaf node is split into two leaf nodes. If the parent node is full, split the parent node, and so on. The most distant leaf node CFs are used as leaf node seeds, with the remaining CFs being assigned to whichever leaf node is closer. Note that the radius of a cluster may be calculated even without knowing the data points, as long as we have the count n , the linear sum LS , and the squared sum SS . This allows BIRCH to evaluate whether a given data point belongs to a particular sub-cluster without scanning the original data set.

Parameters of BIRCH

- **Threshold:** Threshold is the maximum number of data points a sub-cluster in the leaf node of the CF tree can hold.
- **branching_factor:** This parameter specifies the maximum number of CF sub-clusters in each node (internal node).
- **n_clusters:** The number of clusters to be returned after the entire BIRCH algorithm is complete, i.e., the number of clusters after the final clustering step. The final clustering step is not performed if set to none, and intermediate clusters are returned.

Here's an overview of the BIRCH algorithm:

Key Concepts

1. **Clustering Feature (CF):**
 - A triple (N, LS, SS) representing the clustering features of a sub-cluster.
 - N : Number of data points in the sub-cluster.

- **LS:** Linear sum of the data points.
 - **SS:** Square sum of the data points.
 - The CF vector is a compact representation of the cluster and is used to maintain and update the cluster characteristics efficiently.
2. **CF Tree:**
- A height-balanced tree that stores the clustering features.
 - Each non-leaf node contains at most B entries (each entry is a CF).
 - Each leaf node contains at most L entries (each entry is a CF).

Phases of the BIRCH Algorithm

1. **Phase 1: Load the data and build the CF tree**
 - Insert data points into the CF tree, ensuring that the CF tree size remains manageable.
 - If a new data point can be absorbed by an existing leaf node (within a specified threshold), update the corresponding CF. Otherwise, create a new leaf entry.
 - When the tree exceeds the size threshold, apply a split or rebuild strategy to maintain the tree size.
2. **Phase 2: Condense the data**
 - Optionally, further reduce the data by building a smaller CF tree from the initial CF tree.
 - This step helps in improving the efficiency and scalability of the clustering process by reducing the dataset size further.
3. **Phase 3: Global clustering**
 - Apply an existing clustering algorithm (e.g., k-means, agglomerative clustering) to the leaf entries of the CF tree.
 - The CF entries serve as a condensed representation of the original dataset, allowing for efficient clustering.
4. **Phase 4: Refine the clusters**
 - Optionally, refine the clusters by redistributing data points to achieve a better clustering result.
 - This step can help to correct any inaccuracies introduced during the initial CF tree construction.

Advantages of BIRCH

- **Efficiency:** Handles large datasets effectively with a single scan.
- **Incremental and dynamic:** Can process incoming data points dynamically and incrementally.
- **Scalability:** Suitable for very large datasets and can operate in limited memory environments.
- **Hierarchical nature:** Allows multi-level clustering granularity.

Limitations of BIRCH

- **Sensitive to order:** The algorithm can produce different results based on the order of data input.
- **Threshold parameter:** Choosing an appropriate threshold for CF entry splitting is crucial and can be challenging.
- **Not suitable for non-Euclidean data:** The algorithm is designed for metric spaces and may not perform well on non-Euclidean data.

Applications

BIRCH is particularly useful in scenarios where large datasets need to be clustered efficiently and incrementally, such as:

- Data mining and knowledge discovery.
- Image processing and pattern recognition.
- Bioinformatics for clustering gene expression data.
- Market segmentation and customer analysis.

5. The CURE algorithm

The CURE (Clustering Using REpresentatives) algorithm is a hierarchical clustering method designed to handle large datasets efficiently and effectively capture clusters with non-spherical shapes and varying sizes.

Key Concepts:

1. **Hierarchical Clustering:** A method of cluster analysis which seeks to build a hierarchy of clusters.
2. **Representatives:** Points that represent each cluster and are used to determine the shape and spread of the cluster.

Advantages of CURE:

- **Scalability:** By using random sampling and partitioning, CURE can handle large datasets efficiently.
- **Flexibility:** Unlike traditional hierarchical methods, CURE can detect clusters with non-spherical shapes and varying sizes.
- **Robustness:** The shrinking of representative points helps in reducing the effect of outliers.

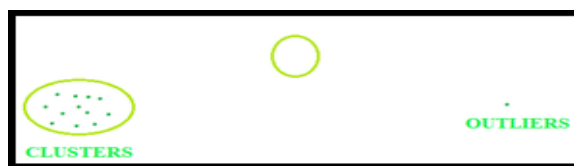
Example to Illustrate:

Imagine a dataset with two distinct clusters: one is elongated (like an ellipse) and the other is more spherical. Traditional clustering methods might struggle to capture the elongated shape. CURE will:

1. Take a random sample from the dataset.
2. Partition the sample and cluster each partition separately.
3. Choose representative points from each partial cluster and shrink them towards their centroids.
4. Use these points to merge clusters hierarchically, capturing the elongated shape effectively.
5. Finally, assign remaining data points to the closest cluster based on the representative points.

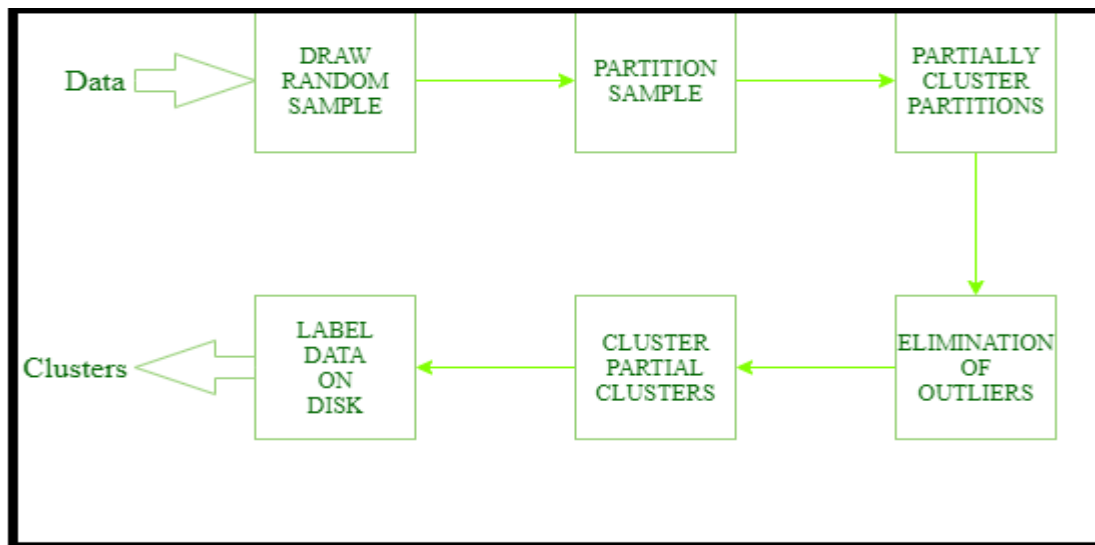
CURE (Clustering Using Representatives)

- It is a hierarchical based clustering technique, that adopts a middle ground between the centroid based and the all-point extremes. Hierarchical clustering is a type of clustering, that starts with a single point cluster, and moves to merge with another cluster, until the desired number of clusters are formed.
- It is used for identifying the spherical and non-spherical clusters.
- It is useful for discovering groups and identifying interesting distributions in the underlying data.
- Instead of using one point centroid, as in most of data mining algorithms, CURE uses a set of well-defined representative points, for efficiently handling the clusters and eliminating the outliers.

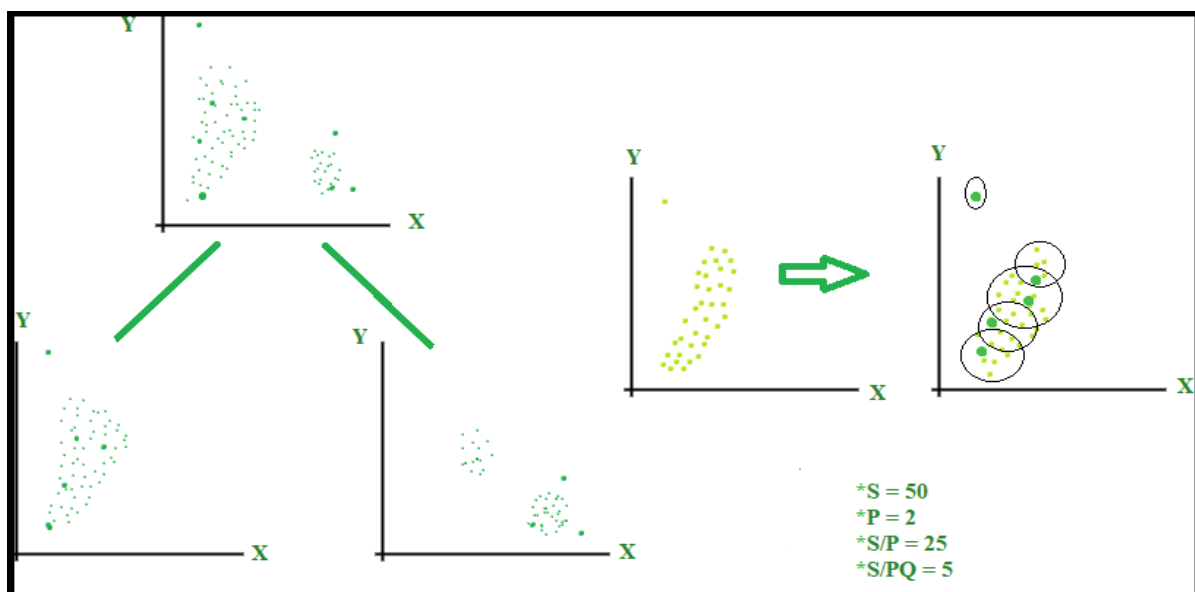


Representation of Clusters and Outliers

Six steps in CURE algorithm:



Idea: Random sample, say 's' is drawn out of a given data. This random sample is partitioned, say 'p' partitions with size s/p . The partitioned sample is partially clustered, into say ' s/pq ' clusters. Outliers are discarded/eliminated from this partially clustered partition. The partially clustered partitions need to be clustered again. Label the data in the disk.



Representation of partitioning and clustering

Procedure :

1. Select target sample number 'gfg'.
2. Choose 'gfg' well scattered points in a cluster.
3. These scattered points are shrunk towards centroid.

4. These points are used as representatives of clusters and used in 'Dmin' cluster merging approach. In Dmin(distance minimum) cluster merging approach, the minimum distance from the scattered point inside the sample 'gfg' and the points outside 'gfg' sample, is calculated. The point having the least distance to the scattered point inside the sample, when compared to other points, is considered and merged into the sample.
5. After every such merging, new sample points will be selected to represent the new cluster.
6. Cluster merging will stop until target, say 'k' is reached.

